

LAPORAN PRATIUM PBO

PEKAN 4



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Bapak Nurfiah

Asisten Praktikum: Uda Dawy Syauqi

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

A. Tahapan Pelaksanaan Praktikum

1. Class Main.java

a. Kode program

```
package Pekan4;
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    // Input PIN sesuai aturan. ch 3
    static String bacaPinBaru (Scanner input, Rekening akunAktif) {
        while (true) {
            System.out.print ("Masukkan PIN (6 digit angka): ");
            String p = input.nextLine();
            String error = Rekening.cekAturanPin (p);
            if (error == null) {
                return p;
            }
            System.out.println ("Gagal: " + error + " Silahkan ulangi.");
        }
    }

    //verifikasi PIN sebelum transaksi. ch 1
    static boolean verifikasiPin(Scanner input, Rekening r) {
        if (r.isTerblokir()) {
            System.out.println("Akun TERBLOKIR. Transaksi tidak dapat dilakukan.");
            return false;
        }
        System.out.print("Masukkan PIN: ");
        String p = input.nextLine();
        if (r.otentikasi(p)) {
            return true;
        }
        if (r.isTerblokir()) {
            System.out.println("PIN salah 3 kali. Akun TERBLOKIR.");
        } else {
            System.out.println("PIN salah! Sisa percobaan: " +
r.getSisaPercobaan());
        }
        System.out.println("Transaksi dibatalkan.");
        return false;
    }
}
```

```

public static void main(String[] args) {

    Scanner input = new Scanner(System.in);

    // Menyimpan banyak objek Rekening
    ArrayList<Rekening> daftarRekening = new ArrayList<>();

    // Menyimpan rekening yang sedang aktif
    Rekening akunAktif = null;

    boolean isRunning = true;

    System.out.println("=== SISTEM PERBANKAN MINI ===");

    while (isRunning) {

        System.out.println("\nMenu Utama:");
        System.out.println("1. Buka Rekening Baru");
        System.out.println("2. Setor Tunai");
        System.out.println("3. Tarik Tunai");
        System.out.println("4. Cek Informasi Rekening");
        System.out.println("5. Ganti Akun");
        System.out.println("6. Cetak Mutasi (Riwayat)");
        System.out.println("7. Ganti PIN");
        System.out.println("8. Simulasi Akhir Bulan");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");

        int pilihan = input.nextInt();
        input.nextLine();

        switch (pilihan) {

            // 1. BUKA REKENING BARU

            case 1:

                System.out.print("Masukkan No Rekening: ");
                String no = input.nextLine();

                // Mengecek apakah nomor rekening sudah digunakan
                boolean sudahAda = false;

                for (Rekening rekening : daftarRekening) {
                    if (rekening.getNomorRekening().equals(no)) {

```

```

        sudahAda = true;
    }
}

if (sudahAda) {
    System.out.println("Gagal: Nomor rekening sudah
digunakan!");
    break;
}

System.out.print("Masukkan Nama Pemilik: ");
String nama = input.nextLine();

System.out.print("Masukkan Saldo Awal: ");
double saldo = input.nextDouble();
input.nextLine ();

String pin = bacaPinBaru(input, null);

System.out.println ("Pilih Produk: ");
System.out.println ("1. Tabungan Umum: ");
System.out.println ("2. Giro Bisnis: ");
System.out.println ("Pilihan Produk: ");
int produk = input.nextInt ();
input.nextLine ();

// Membuat objek Rekening baru
Rekening rekeningBaru;

if (produk == 1) {
    System.out.print ("Masukkan suku bunga (%): ");
    double bunga = input.nextDouble();
    input.nextLine();
    rekeningBaru = new RekeningTabungan (no, nama, saldo,
pin, bunga);
} else if (produk == 2) {
    System.out.print ("Masukkan batas overdraft: ");
    double overdraft = input.nextDouble ();
    input.nextLine();
    rekeningBaru = new RekeningGiro (no, nama, saldo, pin,
overdraft);
} else {
    System.out.println ("Gagal: Pilihan produk tidak valid: ");

```

```

        break;
    }

    // Menambahkan objek ke ArrayList
    daftarRekening.add(rekeningBaru);

    // Menjadikan rekening baru sebagai akun aktif
    akunAktif = rekeningBaru;

    System.out.println("Rekening berhasil ditambahkan ke
daftar.");

    break;

// 2. SETOR TUNAI

case 2:

    if (akunAktif == null) {

        System.out.println(
            "Error: Mohon maaf, Anda belum memiliki rekening!"
        );

    } else {
        // Verifikasi PIN terlebih dahulu
        if (verifikasiPin(input, akunAktif)) {

            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            input.nextLine();

            // Jika PIN benar, baru melakukan setor
            akunAktif.setorTunai(setor);
        }
    }

    break;

// 3. TARIK TUNAI

case 3:

```

```

if (akunAktif == null) {

    System.out.println(
        "Error: Mohon maaf, Anda belum memiliki rekening!"
    );

} else {

    // Verifikasi PIN terlebih dahulu
    if (verifikasiPin(input, akunAktif)) {

        System.out.print("Masukkan nominal tarik: ");
        double tarik = input.nextDouble();
        input.nextLine();

        // Jika PIN benar, baru melakukan penarikan
        akunAktif.tarikTunai(tarik);
    } else {

        System.out.println(
            "Akses Ditolak: PIN yang Anda masukkan salah!"
        );
    }
}

break;

```

// 4. CEK INFORMASI REKENING

case 4:

```

if (akunAktif == null) {

    System.out.println(
        "Error: Anda belum membuka rekening."
    );

} else {

    // Memanggil method cekInformasi()
    akunAktif.cekInformasi();
}

break;

```

```

// 5. GANTI AKUN

case 5:

    if (daftarRekening.isEmpty()) {

        System.out.println(
            "Belum ada rekening yang terdaftar."
        );

    } else {

        System.out.print("Masukkan No Rekening yang ingin
digunakan: ");

        String nomorCari = input.nextLine();

        Rekening rekeningDitemukan = null;

        // Mencari rekening berdasarkan nomor
        for (Rekening rekening : daftarRekening) {

            if (rekening.getNomorRekening().equals(nomorCari)) {
                rekeningDitemukan = rekening;
                break;
            }
        }

        if (rekeningDitemukan != null) {

            akunAktif = rekeningDitemukan;

            System.out.println(
                "Berhasil berganti akun."
            );

            System.out.println(
                "Akun aktif: " + akunAktif.getNamaPemilik()
            );

            if (akunAktif.isTerblokir ()) {
                System.out.println ("Peringatan: akun ini terblokir,
transaksi tidak dapat dilakukan");

            }
        }
    }

```

```

        } else {

            System.out.println(
                "Gagal: Nomor rekening tidak ditemukan!"
            );
        }
    }

    break;

// 6. CETAK MUTASI (RIWAYAT)

case 6:

    if (akunAktif == null) {

        System.out.println(
            "Error: Anda belum membuka rekening."
        );

    } else if (verifikasiPin (input, akunAktif)){
        akunAktif.cetakMutasi ();
    }

    break;

// 7. Ganti PIN. ch 2

case 7: {
    if (akunAktif == null) {

        System.out.println("Error:  Anda  belum  membuka
rekening.");
    } else if (akunAktif.isTerblokir()) {
        System.out.println("Akun  TERBLOKIR.  Tidak  dapat
mengganti PIN.");
    } else {
        System.out.print("Masukkan PIN lama: ");
        String pinLama = input.nextLine();

        if (!akunAktif.otentikasi(pinLama)) {
            if (akunAktif.isTerblokir()) {

```

```

        System.out.println("PIN salah 3 kali. Akun
TERBLOKIR.");
    } else {
        System.out.println("PIN lama salah! Sisa percobaan: "
+ akunAktif.getSisaPercobaan());
    }
    } else {
        String pinBaru = bacaPinBaru(input, akunAktif);
        akunAktif.gantiPin(pinLama, pinBaru);
    }
}
break;
}

// 8. Simulasi Akhir Bulan

case 8:
    if (akunAktif == null) {
        System.out.println ("Error: Anda belum membuka
rekening.");
    } else if (akunAktif instanceof RekeningTabungan) {
        ((RekeningTabungan)
akunAktif).tambahBungaAkhirBulan();
    } else {
        System.out.println ("Simulasi bunga Hanya berlaku
untuk Rekening Tabungan.");
    }
    break;

// 0. KELUAR

case 0:

    isRunning = false;

    System.out.println(
        "Terima kasih telah menggunakan Sistem Perbankan Mini."
    );

    break;

// PILIHAN TIDAK VALID

default:

```

```

        System.out.println(
            "Pilihan menu tidak tersedia!"
        );
    }
}

input.close();
}
}

```

b. Penjelasan

PENJELASAN PROGRAM SISTEM PERBANKAN MINI (Main.java)

Secara garis besar, program ini adalah simulasi ATM atau mesin bank sederhana yang jalan di konsol. Pengguna bisa buka rekening, setor, tarik, cek saldo, ganti akun, lihat riwayat transaksi, ganti PIN, dan simulasi bunga akhir bulan. Class Main ini cuma berperan sebagai "pengatur menu". Logika perhitungan dan aturan rekening ada di class lain (Rekening, RekeningTabungan, RekeningGiro, dan Transaksi).

1. Bagian paling atas

Baris package Pekan4 menandakan semua class ada di satu package yang sama, jadi Main bisa langsung memakai Rekening, RekeningTabungan, RekeningGiro tanpa import tambahan. Import ArrayList dipakai untuk menyimpan banyak rekening, dan import Scanner dipakai untuk membaca input dari keyboard.

2. Method bacaPinBaru

Method ini tugasnya meminta pengguna mengetik PIN baru sampai PIN-nya memenuhi aturan. Isinya perulangan while (true), jadi akan terus minta input sampai ada return. Di dalamnya, PIN yang diketik dikirim ke Rekening.cekAturanPin(p). Method itu mengembalikan null kalau PIN valid, atau pesan error kalau tidak valid. Kalau null,

PIN dikembalikan dan perulangan selesai. Kalau ada pesan error, pesannya dicetak lalu pengguna disuruh ulang. Aturan PIN-nya (ada di class Rekening) adalah harus tepat 6 digit angka, tidak boleh semua angka sama seperti 111111, dan tidak boleh berurutan naik (123456) atau turun (654321). Parameter akunAktif sebenarnya tidak dipakai di dalam method ini, jadi saat membuat rekening baru dia diisi null.

3. Method verifikasiPin

Ini method "satpam" yang dipanggil sebelum transaksi yang butuh PIN. Alurnya begini. Pertama, cek apakah akun terblokir. Kalau iya, langsung tolak dan return false. Kalau tidak, minta pengguna mengetik PIN lalu cocokkan lewat `r.otentikasi(p)`. Kalau benar, return true dan transaksi boleh lanjut. Kalau salah, ada dua kemungkinan pesan. Kalau akun jadi terblokir (salah 3 kali), tampil pesan blokir. Kalau belum, tampil sisa percobaan. Setelah itu transaksi dibatalkan dan return false. Penghitungan salah PIN dan pemblokiran sendiri dilakukan di dalam class Rekening (variabel `gagalBerturut` dan `terblokir`), bukan di Main.

4. Method main: persiapan awal

Di awal ada tiga hal penting. Scanner input dibuat untuk membaca keyboard. ArrayList `daftarRekening` dibuat untuk menyimpan semua rekening yang pernah dibuat. Karena tipenya Rekening, list ini bisa menampung objek `RekeningTabungan` maupun `RekeningGiro` sekaligus, karena keduanya turunan (subclass) dari Rekening. Ini contoh polimorfisme. Lalu ada variabel `akunAktif`, yaitu rekening yang sedang dipakai sekarang, awalnya null karena belum ada rekening. Terakhir ada boolean `isRunning` yang jadi saklar perulangan menu. Selama true, menu terus muncul. Kalau diubah jadi false (lewat menu 0), program berhenti.

5. Perulangan menu dan switch

Di dalam while (isRunning), menu dicetak setiap putaran, lalu program membaca pilihan dengan `input.nextInt()`. Baris `input.nextLine()` sesudahnya wajib ada. Alasannya, `nextInt()` hanya mengambil angkanya dan meninggalkan tombol Enter di buffer. Kalau tidak dibersihkan dengan `nextLine()`, Enter itu akan terbaca sebagai input kosong pada `nextLine()` berikutnya, dan program kelihatan "melompati" pertanyaan. Pola `nextInt` atau `nextDouble` diikuti `nextLine` ini kamu lihat berulang kali di seluruh kode, alasannya sama. Setelah itu, switch (pilihan) memilih blok kode sesuai angka yang diketik.

6. Case 1: Buka Rekening Baru

Ini bagian paling panjang. Urutannya begini. Program minta nomor rekening, lalu mengecek lewat perulangan for-each apakah nomor itu sudah dipakai rekening lain. Kalau sudah, tampil pesan gagal dan break keluar dari case. Kalau belum, lanjut minta nama pemilik dan saldo awal. Berikutnya PIN diminta lewat `bacaPinBaru`, jadi PIN pasti sudah lolos aturan sebelum lanjut. Lalu pengguna memilih produk, 1 untuk Tabungan atau 2 untuk Giro.

Variabel `rekeningBaru` dideklarasikan bertipe `Rekening` tanpa nilai. Kalau memilih 1, program minta suku bunga lalu membuat objek `RekeningTabungan`. Kalau memilih 2, program minta batas overdraft lalu membuat objek `RekeningGiro`. Kalau memilih selain itu, tampil pesan tidak valid lalu break. Java mengizinkan `rekeningBaru` dipakai sesudah blok if tadi karena di semua jalur yang lolos, variabelnya sudah pasti terisi (jalur else berakhir dengan break). Setelah itu rekening dimasukkan ke `daftarRekening`, dijadikan `akunAktif`, dan tampil pesan sukses. Satu hal yang perlu kamu tahu: saat objek dibuat, constructor `Rekening` memvalidasi PIN lagi. Kalau tidak valid, dia

melempar `IllegalArgumentException`. Di sini aman karena PIN sudah divalidasi lewat `bacaPinBaru`.

7. Case 2: Setor Tunai

Pertama dicek apakah `akunAktif` masih null. Kalau iya, tampil pesan belum punya rekening. Kalau ada, panggil `verifikasiPin`. Hanya kalau PIN benar, program minta nominal setor lalu memanggil `akunAktif.setorTunai(setor)`. Di dalam class `Rekening`, setor hanya berhasil kalau nominal lebih dari 0. Saldo ditambah, lalu dibuat catatan Transaksi dengan jenis "Kredit" dan ID unik berbasis waktu (`System.currentTimeMillis`).

8. Case 3: Tarik Tunai

Alurnya mirip setor. Cek akun, verifikasi PIN, minta nominal, lalu panggil `tarikTunai`. Di class `Rekening`, penarikan punya dua syarat: minimal Rp10.000, dan nominal tidak boleh melebihi saldo. Kalau lolos, saldo dikurangi dan dicatat sebagai transaksi "Debit". Perlu diingat, ada blok else di case ini yang mencetak "Akses Ditolak: PIN yang Anda masukkan salah!". Blok itu sebenarnya sudah tidak perlu karena `verifikasiPin` sudah mencetak pesan kegagalannya sendiri, jadi kalau PIN salah, pesannya muncul dua kali. Tidak error, hanya agak mubazir. Boleh dihapus.

9. Case 4: Cek Informasi Rekening

Paling sederhana. Kalau `akunAktif` ada, panggil `cekInformasi()` yang menampilkan nomor rekening, nama pemilik, saldo akhir, dan status (Aktif atau TERBLOKIR). Tidak butuh PIN.

10. Case 5: Ganti Akun

Karena program bisa menyimpan banyak rekening, menu ini dipakai untuk pindah antar rekening. Kalau daftar masih kosong, tampil pesan belum ada rekening. Kalau ada, pengguna mengetik nomor rekening yang dituju, lalu program mencari di `daftarRekening` lewat perulangan. Saat ketemu, rekening

disimpan di rekeningDitemukan dan perulangan dihentikan dengan break. Kalau ketemu, akunAktif diganti, nama pemilik ditampilkan, dan kalau akun itu ternyata terblokir, muncul peringatan. Kalau tidak ketemu, tampil pesan gagal. Perhatikan tidak ada verifikasi PIN di sini, jadi siapa pun bisa berganti akun, PIN baru diminta saat transaksi.

11. Case 6: Cetak Mutasi

Setelah dicek akunnya ada, PIN diverifikasi memakai verifikasiPin (jadi pesan blokir dan sisa percobaan konsisten dengan menu lain). Kalau lolos, cetakMutasi() dipanggil dan mencetak semua isi riwayatTransaksi. Kalau belum ada transaksi, tampil pesan "Belum ada transaksi pada rekening ini".

12. Case 7: Ganti PIN

Urutannya begini. Cek akun ada, lalu cek apakah akun terblokir (kalau iya, tolak). Kalau aman, minta PIN lama dan cocokkan dengan otentikasi. Kalau salah, tampil pesan salah (atau blokir kalau sudah 3 kali). Kalau benar, minta PIN baru lewat bacaPinBaru, lalu panggil gantiPin(pinLama, pinBaru). Di dalam gantiPin masih ada pengecekan lagi, yaitu aturan PIN dan pengecekan riwayatPin supaya PIN baru tidak sama dengan PIN yang pernah dipakai sebelumnya. Case ini diberi kurung kurawal { } sendiri supaya variabel pinLama dan pinBaru punya lingkup sendiri dan tidak bentrok dengan case lain. Catatan kecil: PIN lama sebenarnya diperiksa dua kali (di Main dan di dalam gantiPin). Hasilnya tetap benar, karena kalau PIN lama salah, proses berhenti di Main dan gantiPin tidak sempat dipanggil, jadi kegagalan tidak terhitung double. Pengecekan di Main hanya agak berlebihan. (Koreksi: di jawabanku sebelumnya aku bilang kegagalannya terhitung dua kali, itu keliru.)

13. Case 8: Simulasi Akhir Bulan

Ini fitur bunga yang cuma berlaku untuk tabungan. Kalau `akunAktif` null, tampil error. Kalau `akunAktif` instanceof `RekeningTabungan` bernilai true, berarti objek yang sedang aktif memang rekening tabungan. Lalu dilakukan casting dengan `((RekeningTabungan) akunAktif)` karena variabelnya bertipe `Rekening`, sedangkan method `tambahBungaAkhirBulan()` hanya dimiliki `RekeningTabungan`. Tanpa cast, Java tidak mengenali method itu. Di dalam method tersebut, bunga dihitung dengan saldo dikali `sukuBunga` dibagi 100, ditambahkan ke saldo, dan dicatat sebagai transaksi berjenis "Bunga". Kalau rekeningnya Giro, tampil pesan bahwa bunga hanya untuk Tabungan. Kenapa `RekeningTabungan` bisa langsung mengubah saldo? Karena di class `Rekening`, saldo dan `riwayatTransaksi` bermodifier `protected`, artinya bisa diakses oleh subclass.

14. Case 0 dan default

Case 0 mengubah `isRunning` jadi false dan mencetak pesan terima kasih. Setelah break, putaran while berikutnya tidak jalan lagi karena `isRunning` sudah false, jadi program keluar dari perulangan. Bagian default menangani angka menu yang tidak ada, misalnya 9 atau 15, dengan pesan "Pilihan menu tidak tersedia!".

15. Penutup

Setelah perulangan selesai, `input.close()` menutup Scanner untuk melepas sumber daya, lalu program berakhir.

16. Konsep OOP yang dipakai

Beberapa konsep yang bisa kamu sebut kalau ditanya dosen. Pewarisan (inheritance): `RekeningTabungan` dan `RekeningGiro` mewarisi `Rekening`. Enkapsulasi: atribut seperti `pin`, `nomorRekening`, dan `terblokir` bersifat `private`, diakses lewat method atau getter. Polimorfisme: `ArrayList<Rekening>` menampung dua jenis rekening berbeda, dan `akunAktif` bisa

menunjuk ke keduanya. Casting dan instanceof: dipakai di case 8 untuk mengakses method milik subclass. Method static helper (bacaPinBaru dan verifikasiPin): dipakai supaya kode yang berulang tidak ditulis ulang di tiap case.

17. Hal yang masih rawan

Dua hal kecil. Pertama, kalau pengguna mengetik huruf saat diminta angka (misalnya di menu atau nominal), program akan crash dengan InputMismatchException karena belum ada penanganan error. Kedua, di case 3 ada pesan "Akses Ditolak" yang muncul dobel seperti dijelaskan tadi. Selain itu, ada typo kecil "daapat" di case 5. Tapi dari sisi struktur dan logika utama, program ini sudah jalan dengan benar.

2. Class Rekening

a. Kode program

```
package Pekan4;
import java.util.ArrayList;

public class Rekening {
    private static final int MAKS_GAGAL = 3;
    private int gagalBerturut = 0;
    private boolean terblokir = false;

    private String nomorRekening;
    private String namaPemilik;
    protected double saldo;
    private String pin;
    private ArrayList<String> riwayatPin;
    protected ArrayList<Transaksi> riwayatTransaksi;

    // CONSTRUCTOR

    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        String error = cekAturanPin (pinAwal);
        if (error != null) {
            throw new IllegalArgumentException (error);
        }
    }
}
```

```

        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;
        this.pin = pinAwal;
        this.riwayatPin = new ArrayList<>();
        this.riwayatPin.add (pinAwal);
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println ("Rekening atas nama " + namaPemilik + " berhasil
dibuat");
    }

    // Getter
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }
    public boolean isTerblokir() { return terblokir; }
    public int getSisaPercobaan() { return MAKS_GAGAL - gagalBerturut;
}

    // aturan pembentukan PIN. ch 3 dan 4
    public static String cekAturanPin(String pin) {
        if (pin == null || !pin.matches("\\d{6}")) {
            return "PIN harus terdiri dari tepat 6 digit angka.";
        }

        boolean semuaSama = true;
        boolean urutNaik = true;
        boolean urutTurun = true;

        for (int i = 1; i < pin.length(); i++) {
            char sekarang = pin.charAt(i);
            char sebelum = pin.charAt(i - 1);

            if (sekarang != pin.charAt(0)) semuaSama = false;
            if (sekarang != sebelum + 1) urutNaik = false;
            if (sekarang != sebelum - 1) urutTurun = false;
        }

        if (semuaSama) return "PIN tidak boleh angka yang berulang semua
(contoh 11111).";
        if (urutNaik) return "PIN tidak boleh angka berurutan naik (contoh
123456).";
        if (urutTurun) return "PIN tidak boleh angka berurutan turun (contoh
654321).";
        return null;
    }
}

```

```

// menghitung salah pin + blokir. ch 1
public boolean otentikasi(String inputPin) {
    if (terblokir) {
        return false;
    }
    if (pin.equals(inputPin)) {
        gagalBerturut = 0;
        return true;
    }
    gagalBerturut++;
    if (gagalBerturut >= MAKS_GAGAL) {
        terblokir = true;
    }
    return false;
}

// Ganti pin. Challenge 2
public boolean gantiPin(String pinLama, String pinBaru) {
    if (terblokir) {
        System.out.println("Gagal: akun terblokir.");
        return false;
    }
    if (!otentikasi(pinLama)) {
        if (terblokir) {
            System.out.println("PIN lama salah. Akun TERBLOKIR.");
        } else {
            System.out.println("PIN lama salah! Sisa percobaan: " +
getSisaPercobaan());
        }
        return false;
    }
    String error = cekAturanPin(pinBaru);
    if (error != null) {
        System.out.println("Gagal: " + error);
        return false;
    }
    if (riwayatPin.contains(pinBaru)) {
        System.out.println("Gagal: PIN ini pernah digunakan. Pilih PIN
yang benar-benar baru.");
        return false;
    }
    this.pin = pinBaru;
    riwayatPin.add(pinBaru);
    System.out.println("PIN berhasil diganti.");
    return true;
}

```

```

    }

    // SETOR TUNAI

    public void setorTunai(double nominal) {

        if (nominal > 0) {

            saldo += nominal;

            String idTrx = "TRX-S-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi (idTrx, "Kredit", nominal));
            System.out.println(
                "Setor tunai Rp" + nominal
                + " berhasil. Saldo saat ini: Rp" + saldo
            );

        } else {

            System.out.println(
                "Gagal: Nominal setor harus lebih dari 0!"
            );

        }
    }

    // TARIK TUNAI

    public void tarikTunai(double nominal) {

        // Minimal penarikan Rp10.000
        if (nominal < 10000) {

            System.out.println(
                "Transaksi Gagal: Minimal nominal penarikan Rp10.000."
            );

            // Mengecek apakah saldo mencukupi
        } else if (nominal > saldo) {

            System.out.println(
                "Transaksi Gagal: Saldo tidak mencukupi. "
                + "Saldo Anda: Rp" + saldo
            );

        }
    }
}

```

```

// Jika semua syarat terpenuhi -> penarikan berhasil
} else {

    // Mengurangi saldo
    saldo -= nominal;

    // === Merekam riwayat transaksi penarikan ===
    String idTrx = "TRX-T-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));
    // =====

    System.out.println(
        "Tarik tunai Rp" + nominal
        + " berhasil. Saldo saat ini: Rp" + saldo
    );
}
}

// CEK INFORMASI

public void cekInformasi() {

    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("Status : " + (terblokir ? "TERBLOKIR" :
"Aktif"));
    System.out.println("-----");
}

// METHOD CEK TRANSAKSI TERBARU

public void cekTransaksiTerbaru() {

    if (riwayatTransaksi.size() > 3) {
        Transaksi t = riwayatTransaksi.get(riwayatTransaksi.size() - 1);
        System.out.println("--- TRANSAKSI TERBARU ---");
        System.out.println("ID : " + t.getIdTransaksi());
        System.out.println("Jenis : " + t.getJenis());
        System.out.println("Nominal : Rp" + t.getNominal());
        System.out.println("-----");
    } else {
        System.out.println(

```

```

        "Transaksi belum lebih dari 3");
    }
}

// METHOD CETAK MUTASI

public void cetakMutasi() {

    System.out.println("--- MUTASI REKENING (" + nomorRekening +
") ---");

    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi trx: riwayatTransaksi) {
            trx.cetakDetail();
        }
    }

    System.out.println("-----");
}

} // <-- penutup class Rekening

```

b. Penjelasan

PENJELASAN CLASS REKENING (Rekening.java)

Kalau Main.java adalah "meja kasir" yang menerima perintah dari pengguna, class Rekening adalah "buku tabungan beserta aturannya". Semua data nasabah (nomor, nama, saldo, PIN, riwayat) dan semua aturan (cara setor, tarik, cek PIN, blokir akun) disimpan di sini. Main hanya memanggil method-nya. Class ini juga jadi induk (superclass) bagi RekeningTabungan dan RekeningGiro.

1. Atribut (variabel di dalam class)

Atribut dibagi menjadi tiga kelompok.

Kelompok pengaman PIN:

- MAKS_GAGAL adalah konstanta bernilai 3, batas salah PIN berturut-turut sebelum akun diblokir. Karena static

final, nilainya satu untuk semua rekening dan tidak bisa diubah. Menaruhnya sebagai konstanta lebih rapi daripada menulis angka 3 di banyak tempat.

- gagalBerturut adalah penghitung salah PIN, awalnya 0.
- terblokir menandai status akun. Awalnya false, jadi akun baru dalam keadaan aktif.

Kelompok data nasabah:

- nomorRekening, namaPemilik, dan pin bermodifier private. Artinya hanya bisa diakses dari dalam class Rekening sendiri, bahkan subclass pun tidak bisa. Ini disebut enkapsulasi. Wajar kalau PIN dibuat paling tertutup.
- riwayatPin adalah daftar semua PIN yang pernah dipakai, gunanya untuk mencegah pengguna memakai PIN lama lagi saat ganti PIN.

Kelompok yang boleh diakses subclass:

- saldo dan riwayatTransaksi bermodifier protected. Artinya bisa diakses class itu sendiri dan semua class turunannya (di package yang sama pun bisa). Inilah alasan RekeningTabungan bisa langsung menulis "saldo += nominalBunga" dan "riwayatTransaksi.add(...)" tanpa getter atau setter.

2. Constructor

Constructor dipanggil saat objek dibuat (new Rekening, atau lewat super() dari subclass). Urutannya:

Pertama, PIN awal diperiksa dulu lewat cekAturanPin. Kalau hasilnya bukan null (artinya ada pelanggaran aturan), constructor langsung melempar IllegalArgumentException. Dengan begitu, objek rekening dengan PIN yang tidak valid tidak akan pernah berhasil dibuat. Ini lapisan pengaman kedua, karena di Main sebenarnya PIN sudah divalidasi lewat bacaPinBaru.

Kalau lolos, semua atribut diisi dari parameter. `this.nomorRekening = nomor` artinya "atribut milik objek ini diisi dengan parameter bernama nomor". Kata `this` dipakai untuk membedakan atribut dan parameter kalau namanya sama.

Lalu dua `ArrayList` dibuat: `riwayatPin` dibuat dan langsung diisi PIN awal, sedangkan `riwayatTransaksi` dibuat kosong. Terakhir, tercetak pesan "Rekening atas nama ... berhasil dibuat".

3. Getter

Ada empat getter: `getNomorRekening`, `getNamaPemilik`, `isTerblokir`, dan `getSisaPercobaan`. Dua yang pertama mengembalikan data. `isTerblokir` mengembalikan status blokir. `getSisaPercobaan` menghitung `MAKS_GAGAL` dikurangi `gagalBerturut`, jadi kalau sudah salah 1 kali, sisanya 2.

Perhatikan bahwa tidak ada getter untuk pin dan saldo. PIN sengaja tidak dibuka sama sekali. Saldo hanya bisa dilihat lewat `cekInformasi` dan diubah lewat `setorTunai` atau `tarikTunai`, bukan diubah bebas dari luar. Itu bagian dari prinsip enkapsulasi: data dilindungi, perubahan hanya lewat jalur yang punya aturan.

4. Method `cekAturanPin`

Method ini memeriksa apakah sebuah PIN memenuhi aturan. Kebiasaan pentingnya: kalau valid dia mengembalikan `null`, kalau tidak valid dia mengembalikan teks pesan error. Jadi pemanggil cukup mengecek "`error == null`" untuk tahu valid atau tidaknya, sekaligus punya pesan untuk ditampilkan.

Method ini bermodifikasi `static`, artinya bisa dipanggil tanpa membuat objek, misalnya `Rekening.cekAturanPin(p)` di `Main`. Ini perlu karena PIN harus dicek sebelum rekening ada. Di dalam constructor pun dia dipakai sebelum objek selesai dibuat. Pemeriksaannya bertahap:

Tahap 1, cek format. `pin == null` dicek dulu supaya tidak terjadi error saat memanggil `matches`. Lalu `pin.matches("\\d{6}")`

memakai regular expression: `\d` berarti satu digit angka, `{6}` berarti tepat enam kali. Jadi PIN harus tepat 6 karakter dan semuanya angka. Kalau tidak, langsung return pesan error.

Tahap 2, cek pola. Dibuat tiga penanda boolean yang awalnya true semua: `semuaSama`, `urutNaik`, `urutTurun`. Lalu perulangan berjalan dari indeks 1 sampai akhir, membandingkan setiap digit (sekarang) dengan digit sebelumnya (sebelum). Logikanya "buktikan salah": kalau ada satu saja digit yang melanggar pola, penandanya diubah jadi false.

- Kalau ada digit yang tidak sama dengan digit pertama, berarti bukan "semua sama".
- Kalau ada digit yang bukan sebelum + 1, berarti bukan urutan naik.
- Kalau ada digit yang bukan sebelum - 1, berarti bukan urutan turun.

Soal sebelum + 1: sebelum bertipe char, tapi saat ditambah 1 Java otomatis mengubahnya ke angka kode karakter, dan karakter '0' sampai '9' kodenya berurutan. Jadi '3' + 1 sama dengan kode '4', dan perbandingannya tetap benar.

Setelah perulangan, kalau salah satu penanda masih true, berarti PIN itu melanggar pola dan dikembalikan pesan yang sesuai. Kalau semuanya false, return null artinya PIN lolos. Contohnya 111111 ditolak, 123456 ditolak, 654321 ditolak, tapi 123457 atau 135790 lolos, karena aturannya hanya menolak pola yang benar-benar penuh dari awal sampai akhir.

5. Method otentikasi

Ini jantung sistem keamanan PIN. Dia menerima PIN yang diketik pengguna dan mengembalikan true kalau cocok.

Urutannya: kalau akun sudah terblokir, langsung return false tanpa memeriksa apa pun, jadi PIN yang benar pun tetap ditolak.

Kalau tidak terblokir, PIN dibandingkan dengan

`pin.equals(inputPin)`. Kalau cocok, `gagalBerturut` dikembalikan ke 0 lalu return true. Kata "berturut" penting: satu kali berhasil menghapus catatan gagal sebelumnya. Kalau tidak cocok, `gagalBerturut` ditambah 1, dan kalau sudah mencapai `MAKS_GAGAL` (3) maka terblokir menjadi true, lalu return false.

Method ini tidak mencetak pesan apa pun. Dia hanya menghitung dan memutuskan. Urusan tampilan diserahkan ke pemanggil (`verifikasiPin` di `Main`), jadi logika dan tampilan terpisah dengan rapi.

6. Method `gantiPin`

Method ini mengganti PIN dengan beberapa lapis pemeriksaan, berurutan:

1. Kalau akun terblokir, ditolak.
2. PIN lama diverifikasi lewat otentikasi. Kalau salah, dicetak pesan (atau pesan blokir kalau baru saja mencapai 3 kali salah), lalu return false.
3. PIN baru diperiksa lewat `cekAturanPin`. Kalau melanggar aturan, ditolak dengan pesannya.
4. `riwayatPin.contains(pinBaru)` memeriksa apakah PIN baru pernah dipakai. Kalau pernah, ditolak.
5. Kalau semua lolos, atribut `pin` diganti, PIN baru ditambahkan ke `riwayatPin`, dan tercetak "PIN berhasil diganti". Return true.

Karena ada di dalam class `Rekening`, method ini bebas mengubah atribut private `pin`. Dari luar, tidak ada cara lain untuk mengganti PIN selain lewat method ini, sehingga semua aturan di atas pasti dilewati.

7. Method `setorTunai`

Kalau nominal lebih dari 0, saldo ditambah. Lalu dibuat ID transaksi dengan "TRX-S-" digabung `System.currentTimeMillis()`, yaitu waktu saat ini dalam

milidetik, jadi ID-nya hampir pasti unik. Huruf S berarti Setor. Kemudian objek Transaksi baru dengan jenis "Kredit" dimasukkan ke riwayatTransaksi, dan tercetak pesan sukses beserta saldo terbaru. Kalau nominal nol atau negatif, tercetak pesan gagal dan saldo tidak berubah.

8. Method tarikTunai

Ada tiga kemungkinan, diperiksa berurutan dengan if, else if, else:

- Nominal kurang dari 10.000: gagal, karena minimal penarikan Rp10.000.
- Nominal lebih besar dari saldo: gagal, saldo tidak cukup. Saldo saat ini ikut ditampilkan.
- Selain itu (semua syarat terpenuhi): saldo dikurangi, dibuat ID "TRX-T..." (T untuk Tarik), dicatat sebagai transaksi berjenis "Debit", dan tercetak pesan sukses.

Perlu dicatat, saat ini penarikan tidak pernah boleh melebihi saldo, termasuk untuk RekeningGiro. Batas overdraft di RekeningGiro sudah disimpan tapi belum dipakai, dan menurut komentar di kodenya, itu baru akan dikerjakan di modul berikutnya.

9. Method cekInformasi

Hanya mencetak ringkasan rekening: nomor, nama pemilik, saldo akhir, dan status. Status memakai operator ternary: (terblokir ? "TERBLOKIR" : "Aktif"). Artinya kalau terblokir bernilai true tampil "TERBLOKIR", kalau false tampil "Aktif". Ini versi singkat dari if-else.

10. Method cekTransaksiTerbaru

Method ini mengambil transaksi paling akhir, tapi hanya kalau jumlah transaksi sudah lebih dari 3. riwayatTransaksi.get(size() - 1) mengambil elemen terakhir, karena indeks ArrayList dimulai dari 0. Kalau transaksinya belum lebih dari 3, tercetak

"Transaksi belum lebih dari 3". Syarat "lebih dari 3" ini kemungkinan memang aturan dari soal tugasmu. Perlu diketahui juga bahwa method ini tidak dipanggil dari menu mana pun di Main, jadi sekarang statusnya belum terpakai.

11. Method cetakMutasi

Mencetak judul beserta nomor rekening, lalu mengecek apakah riwayatTransaksi kosong. Kalau kosong, tercetak "Belum ada transaksi pada rekening ini". Kalau ada isinya, perulangan for-each (for (Transaksi trx : riwayatTransaksi)) mengambil satu per satu transaksi dan memanggil trx.cetakDetail(). Cara mencetak detailnya sendiri ditentukan oleh class Transaksi, jadi Rekening tidak perlu tahu formatnya.

12. Hubungan dengan class lain

- Dengan Transaksi: Rekening "memiliki" banyak Transaksi (relasi has-a atau komposisi). Setiap setor, tarik, atau bunga membuat objek Transaksi baru yang disimpan di riwayatTransaksi.
- Dengan RekeningTabungan dan RekeningGiro: keduanya mewarisi (extends) Rekening, sehingga otomatis punya semua atribut dan method di atas, lalu menambah milik sendiri. Constructor mereka memanggil super(nomor, nama, saldoAwal, pinAwal) untuk menjalankan constructor Rekening ini dulu.
- Dengan Main: Main memakai class ini lewat tipe Rekening, dan karena polimorfisme variabel bertipe Rekening bisa menunjuk ke objek tabungan maupun giro.

13. Konsep OOP yang bisa kamu sebut

- Enkapsulasi: atribut sensitif dibuat private, akses lewat method yang punya aturan.
- Modifier protected: dibuka khusus untuk subclass (saldo dan riwayatTransaksi).

- Static: konstanta MAKS_GAGAL dan method cekAturanPin dimiliki oleh class, bukan objek.
- Exception: IllegalArgumentException dilempar di constructor kalau PIN tidak valid.
- Komposisi: Rekening menyimpan kumpulan objek Transaksi.
- Pewarisan: jadi superclass untuk RekeningTabungan dan RekeningGiro.

3. Class RekeningGiro

a. Kode program

```
package Pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro (String nomor, String nama, double saldoAwal, String
pinAwal, double batasOverdraft)
    { // Memanggil inialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
    // (Catatan: Penarikan hingga limit overdraft akan kita selesaikan di modul
5)
}
```

b. Penjelasan

PENJELASAN CLASS REKENINGGIRO (RekeningGiro.java)

Class ini adalah jenis rekening khusus untuk keperluan bisnis. Isinya sangat pendek karena hampir semua fungsi dasar (setor, tarik, cek PIN, blokir, riwayat) sudah ada di class induknya,

yaitu Rekening. RekeningGiro hanya menambahkan satu hal yang membedakannya dari rekening biasa: batas overdraft.

1. Deklarasi class

Baris "public class RekeningGiro extends Rekening" artinya RekeningGiro adalah anak (subclass) dari Rekening. Kata extends inilah yang disebut pewarisan atau inheritance. Dengan begitu RekeningGiro otomatis punya semua method yang bisa diakses dari Rekening, seperti setorTunai, tarikTunai, cekInformasi, cetakMutasi, otentikasi, dan gantiPin, tanpa perlu menulis ulang. Hubungannya bisa dibaca sebagai "Rekening Giro adalah sebuah Rekening" (relasi is-a).

2. Atribut batasOverdraft

Atribut ini bertipe double dan bermodifier private. Overdraft adalah fasilitas yang membolehkan nasabah menarik uang melebihi saldo yang dimilikinya, sampai batas tertentu. Misalnya saldo Rp1.000.000 dan batas overdraft Rp500.000, maka secara teori nasabah boleh menarik sampai Rp1.500.000, dan saldonya jadi minus Rp500.000. Atribut ini hanya dimiliki RekeningGiro. Rekening biasa dan RekeningTabungan tidak punya, itulah yang disebut atribut spesifik subclass.

Karena private, atribut ini hanya bisa diakses dari dalam RekeningGiro sendiri. Kelas lain, termasuk Main, harus lewat getter.

3. Constructor

Constructor menerima lima parameter: nomor, nama, saldoAwal, pinAwal, dan batasOverdraft. Empat parameter pertama adalah data dasar rekening, sedangkan yang kelima khusus Giro.

Baris pertama di dalamnya adalah super(nomor, nama, saldoAwal, pinAwal). Ini memanggil constructor class induk (Rekening) untuk mengisi data dasar: nomor rekening, nama

pemilik, saldo, PIN, serta membuat riwayatPin dan riwayatTransaksi. Aturan Java mewajibkan pemanggilan `super()` berada di baris paling pertama constructor. Alasannya masuk akal: bagian "induk" dari objek harus selesai dibangun lebih dulu sebelum bagian "anak" ditambahkan.

Perlu diingat juga, validasi PIN ikut terjadi di sini. Karena `super()` memanggil constructor `Rekening`, PIN yang tidak sesuai aturan akan membuat `IllegalArgumentException` dilempar, sehingga objek `RekeningGiro` tidak jadi dibuat. Di program utama hal itu aman karena PIN sudah divalidasi lewat `bacaPinBaru`.

Setelah `super()` selesai, baru baris `this.batasOverdraft = batasOverdraft` dijalankan. Kata `this` dipakai untuk membedakan atribut milik objek dari parameter yang namanya sama.

4. Getter `getBatasOverdraft`

Method ini hanya mengembalikan nilai `batasOverdraft`. Karena atributnya `private`, getter inilah satu-satunya jalan bagi class lain untuk membacanya. Ini contoh enkapsulasi. Perhatikan tidak ada setter, jadi `batas overdraft` hanya bisa ditentukan saat rekening dibuat dan tidak bisa diubah dari luar sesudahnya.

5. Catatan di bagian bawah

Komentar terakhir menyebutkan bahwa penarikan sampai limit overdraft akan diselesaikan di modul 5. Artinya, saat ini `batasOverdraft` sudah tersimpan tetapi belum berpengaruh apa-apa. Method `tarikTunai` yang dipakai `RekeningGiro` masih method milik `Rekening`, dan di sana penarikan tetap ditolak kalau nominal lebih besar dari saldo. Jadi untuk sekarang, rekening Giro berperilaku sama persis dengan rekening biasa, hanya saja menyimpan angka `batas overdraft`.

Kemungkinan besar di modul 5 kamu akan memakai method overriding, yaitu menulis ulang `tarikTunai` di dalam

RekeningGiro (dengan @Override) supaya syaratnya menjadi "nominal tidak boleh melebihi saldo ditambah batasOverdraft". Itu juga contoh polimorfisme: saat akunAktif menunjuk ke objek Giro, memanggil akunAktif.tarikTunai() akan menjalankan versi milik Giro, walaupun variabelnya bertipe Rekening. Satu hal yang perlu diperhatikan nanti: variabel saldo di Rekening bermodifier protected, jadi RekeningGiro bisa langsung mengubahnya.

6. Perbandingan dengan RekeningTabungan

Kedua subclass punya pola yang sama: extends Rekening, punya satu atribut khusus (sukuBunga untuk Tabungan, batasOverdraft untuk Giro), dan constructor yang memanggil super() dulu. Bedanya, RekeningTabungan sudah punya method tambahan yang berfungsi (tambahBungaAkhirBulan), sedangkan RekeningGiro baru punya atribut dan getter saja.

4. Class RekeningTabungan

a. Kode program

```
package Pekan4;

public class RekeningTabungan extends Rekening{
    //Atribut spesifik yang hanya dimiliki oleh Tabungan

    private double sukuBunga;

    //Constructor Subclass
    public RekeningTabungan (String nomor, String nama, double saldoAwal,
String pinAwal, double sukuBunga) {
        //super () memanggil constructor kelas induk (Rekening). Wajib
berada di baris pertama!
        super (nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan () {
        //Menghitung bunga
```

```
//Mengapa bisa mengakses saldo secara langsung dari class
RekeningTabungan?
double nominalBunga = saldo * (sukuBunga / 100);
saldo += nominalBunga;

//Mencatat riwayat transaksi
String idTrx = "TRX-B-" + System.currentTimeMillis();
riwayatTransaksi.add (new Transaksi (idTrx, "Bunga",
nominalBunga));

System.out.println ("Bunga " + sukuBunga + "% berhasil
ditambahkan: Rp" + nominalBunga);

    }
}
```

c. Penjelasan

*PENJELASAN CLASS REKENINGTABUNGAN
(RekeningTabungan.java)*

Class ini adalah jenis rekening untuk menabung, yang punya fitur khas yaitu bunga. Sama seperti RekeningGiro, isinya pendek karena fungsi dasar (setor, tarik, cek PIN, blokir, riwayat) sudah diwariskan dari class Rekening. RekeningTabungan hanya menambahkan hal yang membedakannya: suku bunga dan method untuk menghitung bunga akhir bulan.

1. Deklarasi class

Baris "public class RekeningTabungan extends Rekening" artinya RekeningTabungan adalah anak (subclass) dari Rekening. Lewat kata extends, dia otomatis punya semua method yang bisa diakses dari induknya, seperti setorTunai, tarikTunai, cekInformasi, cetakMutasi, otentikasi, dan gantiPin. Hubungannya bisa dibaca "Rekening Tabungan adalah sebuah Rekening" (relasi is-a).

2. Atribut sukuBunga

Atribut ini bertipe double dan private. Isinya persentase bunga per bulan, misalnya 2.5 berarti 2,5%. Atribut ini hanya dimiliki RekeningTabungan, sedangkan Rekening biasa dan RekeningGiro tidak punya. Karena private, hanya method di dalam class ini yang bisa membacanya. Perlu dicatat, di class ini belum ada getter untuk sukuBunga, jadi class lain (termasuk Main) tidak bisa melihat nilainya. Kalau nanti dibutuhkan, tinggal tambahkan getSukuBunga().

3. Constructor

Constructor menerima lima parameter: nomor, nama, saldoAwal, pinAwal, dan sukuBunga. Empat yang pertama adalah data dasar rekening, yang kelima khusus tabungan.

Baris pertamanya adalah `super(nomor, nama, saldoAwal, pinAwal)`, yang memanggil constructor Rekening untuk mengisi data dasar dan menyiapkan riwayatPin serta riwayatTransaksi. Java mewajibkan `super()` berada di baris paling pertama, karena bagian "induk" dari objek harus selesai dibangun dulu sebelum bagian "anak" ditambahkan. Seperti di Giro, validasi PIN ikut berjalan di sini, jadi PIN yang melanggar aturan membuat `IllegalArgumentException` dilempar dan objek tidak jadi dibuat. Setelah itu baru `this.sukuBunga = sukuBunga` dijalankan. Kata `this` membedakan atribut milik objek dari parameter yang namanya sama.

4. Method tambahBungaAkhirBulan

Ini satu-satunya method baru di class ini. Langkahnya:

Pertama, bunga dihitung dengan rumus $\text{nominalBunga} = \text{saldo} * (\text{sukuBunga} / 100)$. Contohnya saldo Rp1.000.000 dan sukuBunga 2, maka bunganya $1.000.000 \times 0,02 = \text{Rp}20.000$. Pembagian 100 dipakai karena sukuBunga disimpan dalam bentuk persen, bukan desimal.

Kedua, saldo ditambah bunga lewat `saldo += nominalBunga`.

Ketiga, dibuat ID transaksi "TRX-B-" digabung `System.currentTimeMillis()`, yaitu waktu sekarang dalam milidetik, jadi ID-nya hampir pasti unik. Huruf B berarti Bunga. Lalu objek Transaksi baru dengan jenis "Bunga" dimasukkan ke `riwayatTransaksi`, sehingga muncul di cetak mutasi.

Keempat, tercetak pesan yang menampilkan persentase bunga dan nominal yang ditambahkan.

Method ini dipanggil dari menu 8 (Simulasi Akhir Bulan) di Main, lewat casting ((`RekeningTabungan`) `akunAktif`).`tambahBungaAkhirBulan()`. Casting itu perlu karena variabel `akunAktif` bertipe `Rekening`, sedangkan method ini hanya ada di `RekeningTabungan`.

5. Hal yang perlu diperhatikan

Method ini tidak punya pengecekan apa pun, jadi bunga akan ditambahkan setiap kali menu 8 dipilih, tidak peduli sudah berapa kali dipanggil di bulan yang sama. Itu wajar karena namanya "simulasi". Kalau saldo nol, bunganya juga nol, dan tetap tercatat sebagai transaksi bunga Rp0. Itu tidak error, hanya mungkin kurang rapi kalau mau dibuat lebih realistis. Selain itu, tidak ada validasi untuk sukuBunga negatif, jadi kalau pengguna memasukkan angka negatif saat membuat rekening, "bunga" justru akan mengurangi saldo.

5. Class Transaksi

a. Kode program

```
package Pekan4;

public class Transaksi {
    //1. mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;
```

```

//Constructor
public Transaksi (String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}

//2. hanya menyediakan getter (Read-only)
public String getIdTransaksi () {return idTransaksi;}
public String getJenis () { return jenis;}
public double getNominal () {return nominal;}

public void cetakDetail () {
    System.out.println ("ID: " + idTransaksi + " | Jenis: " + jenis
+ " | Nominal: Rp" +nominal);
}
}

```

b. Penjelasan

PENJELASAN CLASS TRANSAKSI (Transaksi.java)

Kalau Rekening adalah buku tabungan, maka Transaksi adalah satu baris catatan di dalam buku itu. Setiap kali ada setor, tarik, atau bunga, program membuat satu objek Transaksi yang menyimpan "apa yang terjadi, berapa nominalnya, dan nomor catatannya". Class ini paling sederhana di antara kelima class karena hanya menyimpan data dan mencetaknya. Tidak ada perhitungan atau aturan rumit di dalamnya.

1. Atribut

Ada tiga atribut, semuanya private:

- idTransaksi (String): nomor unik transaksi, misalnya "TRX-S-1728291034567". Awalan S, T, dan B menunjukkan jenisnya (Setor, Tarik, Bunga), diikuti waktu dalam milidetik yang dibuat di class Rekening dan RekeningTabungan.
- jenis (String): kategori transaksi. Nilai yang dipakai di program ini adalah "Kredit" (setor), "Debit" (tarik), dan "Bunga".
- nominal (double): jumlah uang pada transaksi itu.

Komentar "1. mengubah semua atribut menjadi private" di kodemu menunjukkan tujuannya: enkapsulasi. Data tidak bisa dibaca atau diubah langsung dari luar, hanya lewat method yang disediakan class ini.

2. Constructor

Constructor menerima tiga parameter (id, jenis, nominal) lalu mengisi ketiga atribut. Kata `this` dipakai untuk membedakan atribut milik objek dari parameter. Misalnya pada `this.jenis = jenis`, sisi kiri adalah atribut, sisi kanan adalah parameter. Pada `this.idTransaksi = id` tidak ada bentrok nama, tapi pemakaian `this` tetap benar dan konsisten.

Constructor ini dipanggil dari tiga tempat, dengan pola "`new Transaksi(idTrx, \"Kredit\", nominal)`" di `setorTunai`, "`new Transaksi(idTrx, \"Debit\", nominal)`" di `tarikTunai`, dan "`new Transaksi(idTrx, \"Bunga\", nominalBunga)`" di `tambahBungaAkhirBulan`.

3. Getter (read-only)

Ada tiga getter: `getIdTransaksi`, `getJenis`, dan `getNominal`. Komentar "2. hanya menyediakan getter (Read-only)" menjelaskan rancangannya. Karena tidak ada setter, begitu objek `Transaksi` dibuat, isinya tidak bisa diubah dari luar. Ini masuk akal untuk catatan transaksi: riwayat bank tidak boleh bisa diedit sesudah tercatat. Sifat ini sering disebut `immutable` (tidak berubah) atau setidaknya `read-only` dari sisi luar.

Getter ini dipakai oleh method `cekTransaksiTerbaru` di class `Rekening` untuk menampilkan ID, jenis, dan nominal transaksi terakhir.

4. Method cetakDetail

Method ini mencetak satu baris berformat: ID, jenis, dan nominal yang digabung dengan tanda `"|"` sebagai pemisah, misalnya:

ID: TRX-S-1728291034567 | Jenis: Kredit | Nominal: Rp50000.0

Method ini dipanggil oleh cetakMutasi di class Rekening dalam perulangan, sekali untuk tiap transaksi. Ini bagus untuk pembagian tugas: Rekening tidak perlu tahu cara memformat satu transaksi, cukup memanggil `trx.cetakDetail()`. Kalau nanti formatnya ingin diubah, cukup ubah di satu tempat ini.

Karena nominal bertipe double, hasil cetaknya tampil dengan ".0" di belakang (50000.0). Kalau mau lebih rapi, bisa memakai `System.out.printf` dengan format seperti `"%.2f"`, atau `String.format`.

5. Hubungan dengan class lain

- Dengan Rekening: relasinya has-a (komposisi). Satu Rekening menyimpan banyak Transaksi di `ArrayList riwayatTransaksi`. Transaksi sendiri tidak tahu menahu soal Rekening, jadi class ini berdiri sendiri dan tidak punya ketergantungan ke class lain. Itu desain yang bersih.
- Dengan `RekeningTabungan` dan `RekeningGiro`: tidak berhubungan langsung. Mereka memakainya lewat `riwayatTransaksi` yang diwariskan dari Rekening (`RekeningTabungan` membuat Transaksi "Bunga" sendiri).
- Dengan Main: tidak ada hubungan langsung. Main hanya memanggil `cetakMutasi`, dan Rekening yang mengurus pencetakan Transaksi.

6. Program lainnya

Enkapsulasi (atribut private, akses lewat getter), objek read-only/immutable (tanpa setter), komposisi (Rekening memiliki banyak Transaksi), dan tanggung jawab tunggal (class ini hanya mengurus data dan tampilan satu transaksi).

Output

```
Main (3) [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full
a 1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
pu 4. Cek Informasi Rekening
Ba 5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
8. Simulasi Akhir Bulan
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: sese
Masukkan Saldo Awal: 45000
Masukkan PIN (6 digit angka): 767654
lu 1. Tabungan Umum:
gT 2. Giro Bisnis:
ah Pilihan Produk:
H 1
Masukkan suku bunga (%): 23
Rekening atas nama sese berhasil dibuat
Rekening berhasil ditambahkan ke daftar.

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
te 5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
8. Simulasi Akhir Bulan
0. Keluar
Pilih menu: 8
Bunga 23.0% berhasil ditambahkan: Rp10350.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Ganti PIN
8. Simulasi Akhir Bulan
0. Keluar
Pilih menu:
```

B. Latihan/Tugas

1. Tugas 1: Pembuatan Multi-Produk (40%)

Kode program overall di setiap class sudah lengkap, yang perlu ditukar atau ditambahkan adalah pada class Main.java

```

        break;
    }

    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();

    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();

    String pin = bacaPinBaru(input, null);

    System.out.println ("Pilih Produk: ");
    System.out.println ("1. Tabungan Umum: ");
    System.out.println ("2. Giro Bisnis: ");
    System.out.print ("Pilihan Produk: ");
    int produk = input.nextInt ();
    input.nextLine ();

    // Membuat objek Rekening baru
    Rekening rekeningBaru;

    if (produk == 1) {
        System.out.print ("Masukkan suku bunga (%): ");
        double bunga = input.nextDouble();
        input.nextLine();
        rekeningBaru = new RekeningTabungan (no, nama, saldo, pin, bunga);
    } else if (produk == 2) {
        System.out.print ("Masukkan batas overdraft: ");
        double overdraft = input.nextDouble ();
        input.nextLine();
        rekeningBaru = new RekeningGiro (no, nama, saldo, pin, overdraft);
    } else {
        System.out.println ("Gagal: Pilihan produk tidak valid! ");
        break;
    }
}

```

2. Tugas 2: Simulasi Waktu Akhir Bulan : 60%)

Kode program overall di setiap class sudah lengkap, yang perlu ditukar atau ditambahkan adalah pada class Main.java

```

16         break;
17     }
18
19     // 8. Simulasi Akhir Bulan
20
21     case 8:
22         if (akunAktif == null) {
23             System.out.println ("Error: Anda belum membuka rekening.");
24         } else if (akunAktif instanceof RekeningTabungan) {
25             RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
26             tabungan.tambahBungaAkhirBulan();
27         } else {
28             System.out.println ("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
29         }
30         break;
31
32     // 0. KELUAR
33

```

C. Link repository Github

https://github.com/shofwatulhasanahsarma/Prak-PBO-_A_2511533015/tree/Praktikum%2C-LapPrak-dan-Tugas-Pekan-4